User Acceptance Testing (UAT) Execution Document

Date	29 October 2026
Team ID	
Project Name	FreelanceHub — MERN Stack Freelancing Platform
Maximum Marks	4 Marks

1. Project Overview & Testing Environment

Project Name: FreelanceHub: A MERN Stack Freelancing Platform

Project Description: A full-stack MERN application that connects clients with freelancers, featuring JWT-based authentication with role-based access (Client, Freelancer, Admin), project posting and browsing, an application workflow with approve/reject status tracking, and real-time chat powered by Socket.io.

Project Version: v1.0.0

Testing Period: [start date] to [end date]

Testing Scope:

- User registration, login, and JWT-protected route access with role-based authorization.
- Client project posting, and freelancer project browsing and filtering.
- Application submission, client review, approve/reject, and status tracking for freelancers.
- Real-time chat between an approved client and freelancer, API testing in Postman, and form validation and error handling.

Testing Environment:

• URL/Location: Local development environment (http://localhost:5173 client, http://localhost:5000 API)

• Credentials: Client: [test email / password] | Freelancer: [test email / password]

2. Executed User Acceptance Test Cases

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Pass/Fail
TC-001	User Registration	1. Go to /register. 2. Enter name, email and password. 3. Choose the role (Client or Freelancer). 4. Submit.	Account is created with the chosen role and the password is stored hashed (bcrypt).	Account created successfully with the chosen role; password stored in hashed form.	**Pass**
TC-002	User Login	1. Go to /login. 2. Enter valid email and password. 3. Submit.	User is authenticated, receives a valid JWT and is redirected to the dashboard for their role.	Login succeeded; JWT returned and the user landed on the correct role dashboard.	**Pass**
TC-003	Role-based Access Control	1. Log in as a Freelancer and try to post a project. 2. Log in as a Client and try to submit an application.	Both actions are blocked with an access denied response.	Freelancer could not post projects and Client could not submit applications; access was denied.	**Pass**

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Pass/Fail
TC-004	Post a Project (Client)	1. Log in as a Client. 2. Open the Post Project form. 3. Enter title, description, budget, duration and skills. 4. Submit.	Project is created and appears in the client's My Projects list.	Project created and listed under My Projects with all entered details.	**Pass**
TC-005	Browse & Filter Projects (Freelancer)	1. Log in as a Freelancer. 2. Open Browse Projects. 3. Apply a skill filter.	All open projects are listed and the filter narrows them to matching projects.	Open projects displayed and the filter returned the expected matches.	**Pass**
TC-006	Submit Application	1. Click Apply on a project. 2. Enter proposal, rate and portfolio link. 3. Submit. 4. Open My Applications.	Application is saved with status Pending and shown in My Applications.	Application submitted; it appeared in My Applications with status Pending.	**Pass**
TC-007	View Applications (Client)	1. Log in as the Client who posted the project. 2. Open View Applications for that project.	All applications for the project are shown with proposal, rate and portfolio link.	All received applications displayed with their proposal, rate and portfolio link.	**Pass**
TC-008	Approve / Reject Application	1. As the Client, approve one application and reject the others. 2. Log in as the Freelancer and open My Applications.	Statuses change to Approved and Rejected and are reflected on the freelancer's page.	Status changes saved and shown correctly on the freelancer's My Applications page.	**Pass**
TC-009	Real-time Chat	1. After approval, open the Chat page as both Client and Freelancer. 2. Send messages from each side.	Messages appear instantly on both screens without refreshing and are saved in the database.	Messages appeared instantly on both sides and were stored in the Chat collection.	**Pass**
TC-010	API Token Protection	1. In Postman, call a protected endpoint without a token. 2. Call it with an invalid token. 3. Call it with a valid token.	Missing or invalid token returns 401; a valid token is accepted.	Requests without or with an invalid token were rejected with 401; the valid token worked.	**Pass**
TC-011	Validation & Error Handling	1. Submit empty forms. 2. Log in with a wrong password. 3. Register with an existing email.	Proper error messages are shown for each case and no invalid data is saved.	Appropriate error messages (toast notifications) appeared for all three cases.	**Pass**

3. System Defect Register & Bug Tracking

Defects genuinely encountered and resolved during development are recorded here as an honest record of the debugging process.

Bug ID	Bug Description	Steps to Reproduce	Severity	Status	Resolution
BG001	[Describe a real bug you hit while building, e.g. a server crash or a wrong API response]	[Steps to reproduce]	[High / Medium / Low]	[Closed]	[How you fixed it]

Bug ID	Bug Description	Steps to Reproduce	Severity	Status	Resolution
BG002	[Describe a second bug, if any]	[Steps to reproduce]	[Severity]	[Closed]	[How you fixed it]
BG003	[Describe a third bug, if any]	[Steps to reproduce]	[Severity]	[Closed]	[How you fixed it]

4. Formal Acceptance Sign-Off

Tester: Mansi Wagh (project developer), self-tested against all executed test cases above

Date of Test Completion: [date]

Notes:

- All core user flows (registration through chat) and the client and freelancer flows (posting, applying, approving, rejecting) have been manually tested end to end.
- All identified defects have been resolved and re-verified.
- Known limitations: online payments and escrow, file uploads, ratings and reviews, email notifications and a full admin panel are not yet implemented; the application is not yet deployed to a public URL.